Perlin Noise

Procedural generation basics for terrains, caves, and biomes

WHAT IS NOISE?

Concept

- Noise returns **pseudo-random values** based on position. The result has some sort of paterns that are usefull.
- Typically outputs values in **[0, 1]** (black → white). Libraries sometimes output [-1 and 1] so be carefull!
- Used to generate **mountains, caves, terrain, biomes**.
- Deterministic: same input → same output.

Noise is very uniform and scalable, but harder to control than hand-crafted generation.

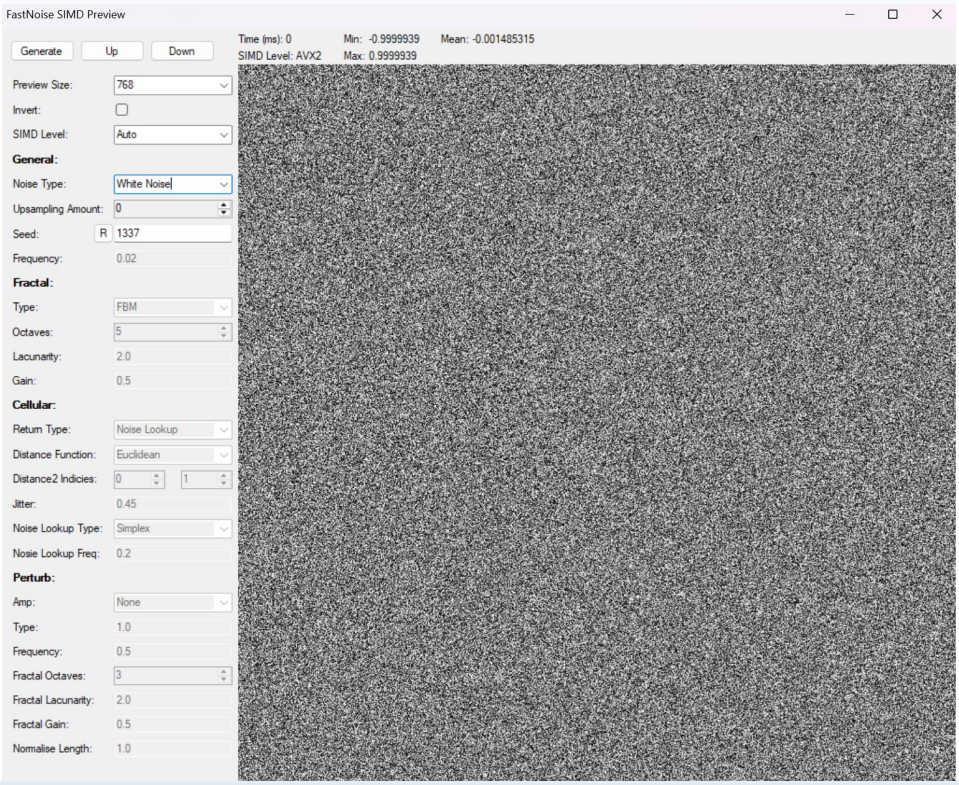
WHITE NOISE

Noise Type

- Completely random values.
- No smoothness, no structure.

Usecase example: get a different starting seed for each world position that you can later use to generate other random things like the loot in a chest.

White Noise



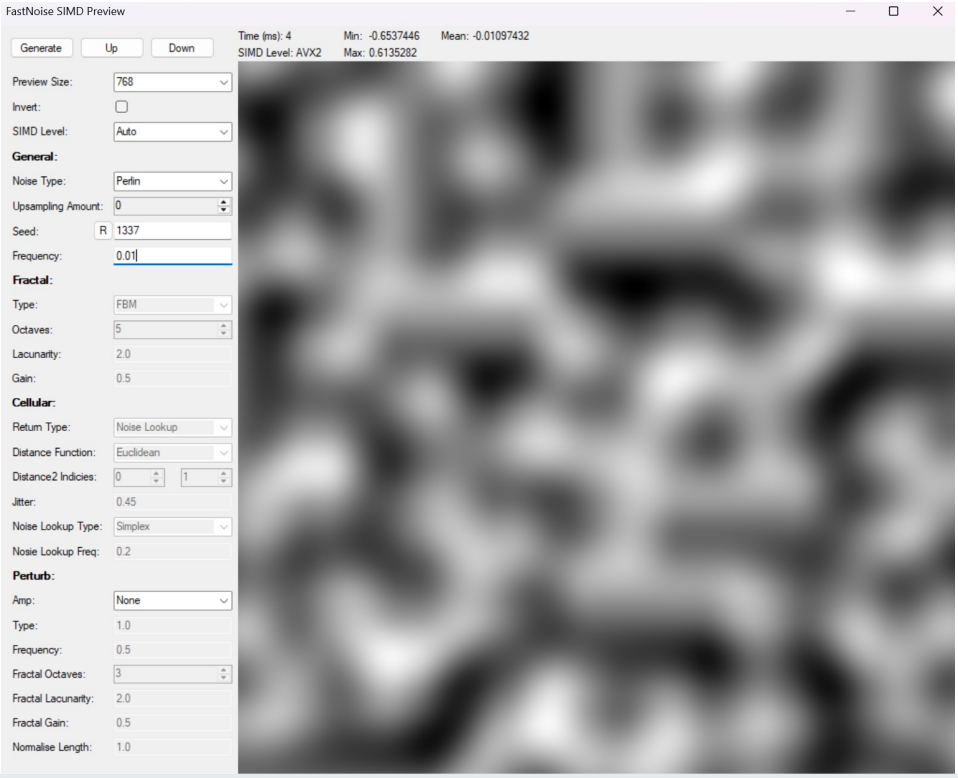
PERLIN NOISE

Noise Type

- Smooth, continuous noise.
- Values change gradually over space.

Thresholding Perlin noise (e.g. value < 0.1) can generate cave-like patterns.

Perlin Noise

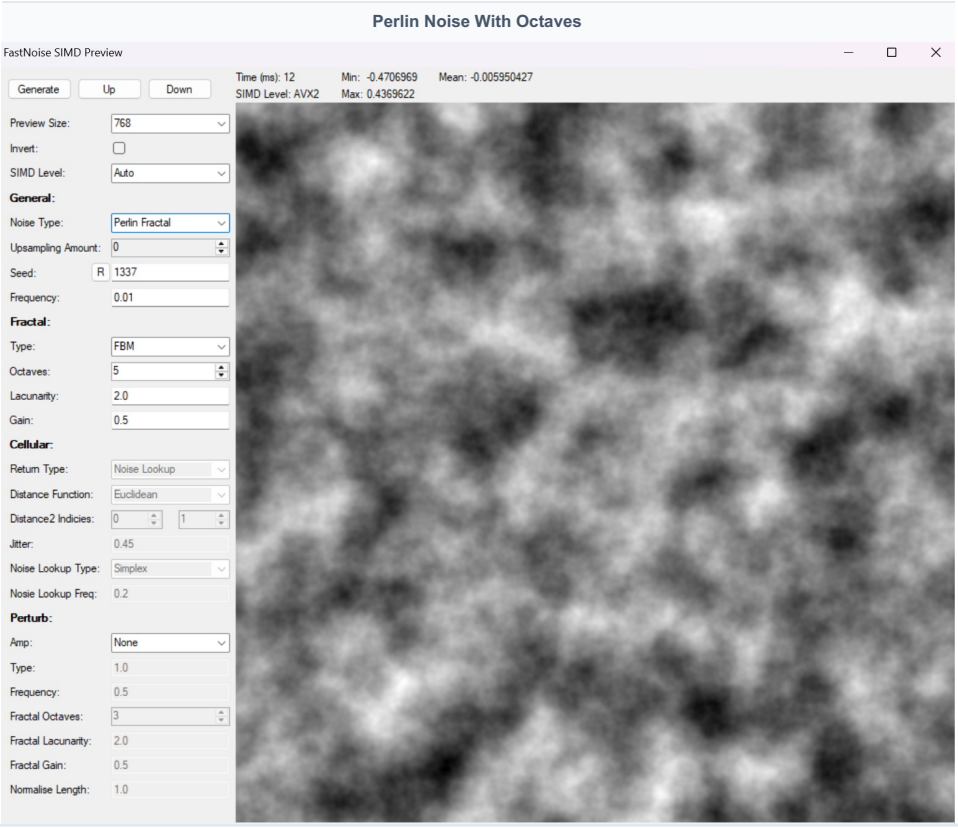


PERLIN FRACTAL (OCTAVES)

Noise Type

- Multiple layers of Perlin noise.
- **Octaves** = number of layers.
- More octaves → more detail.

Each octave adds cost. Don't blindly stack many layers.



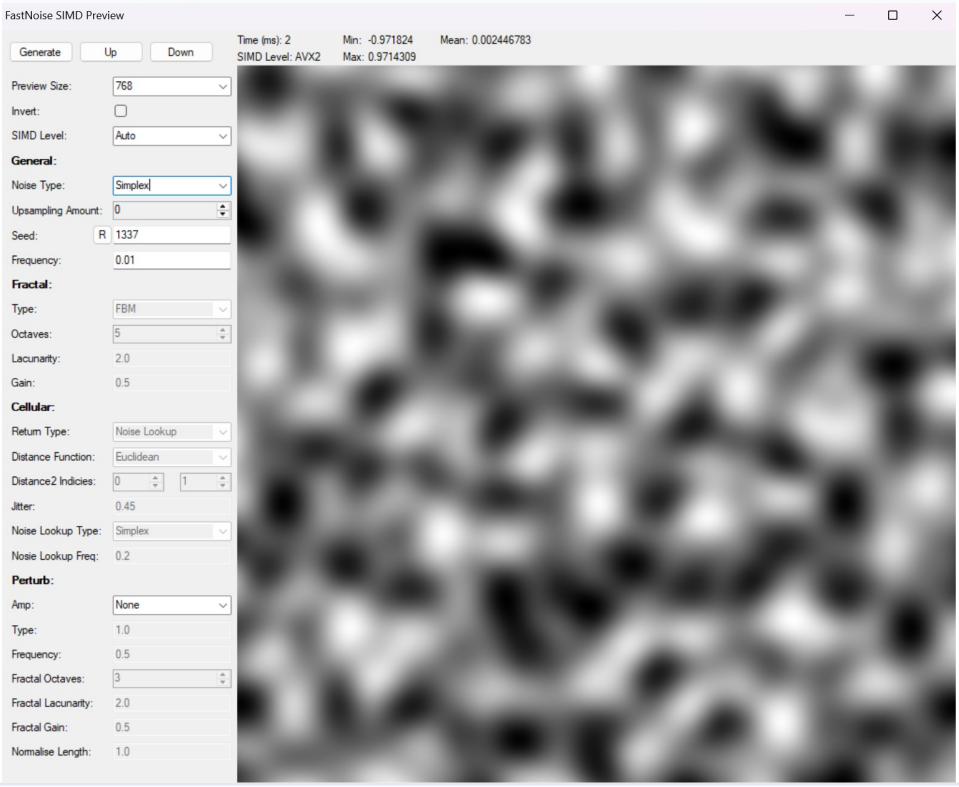
SIMPLEX NOISE

Noise Type

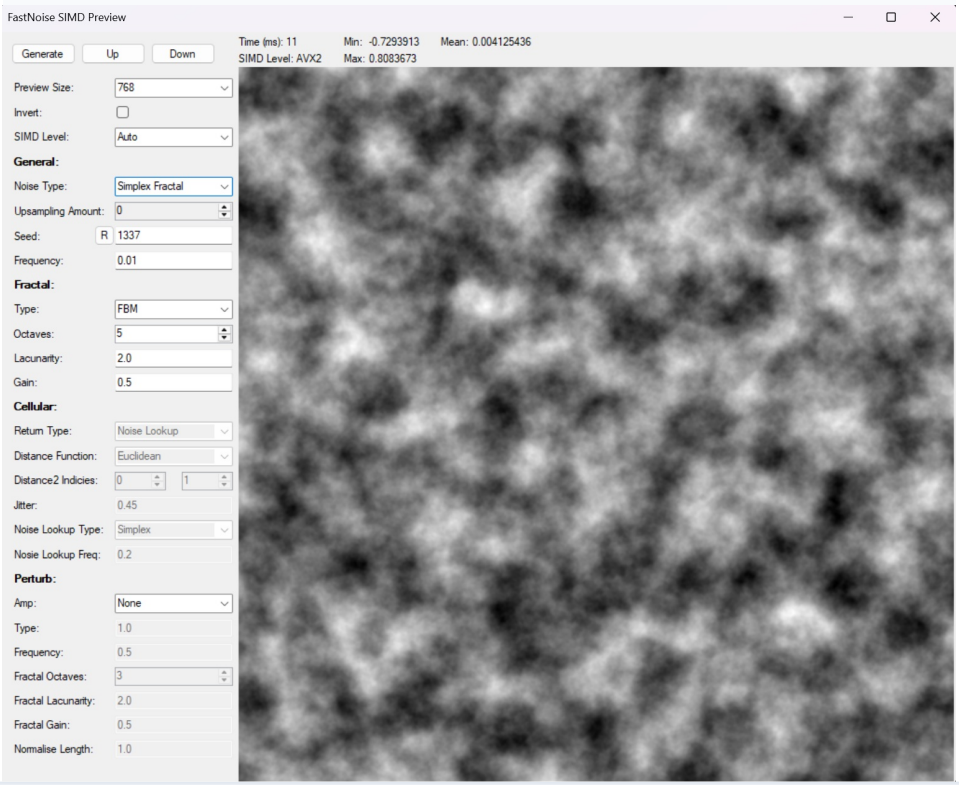
- Similar to Perlin, but higher quality.
- No visible grid or straight-edge artifacts.
- Preferred choice in most cases.

Simplex Fractal = Simplex noise with octaves, same as it was with Perlin.

Simplex Noise



Simplex Fractal (Octaves)



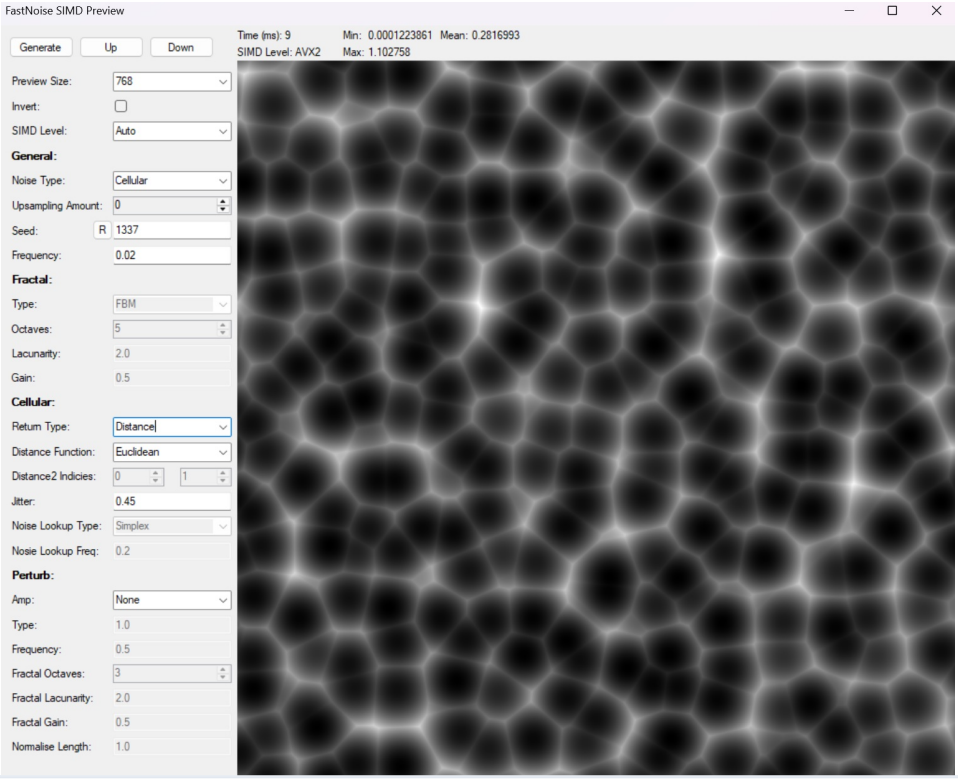
CELLULAR (VORONOI) NOISE

Noise Type

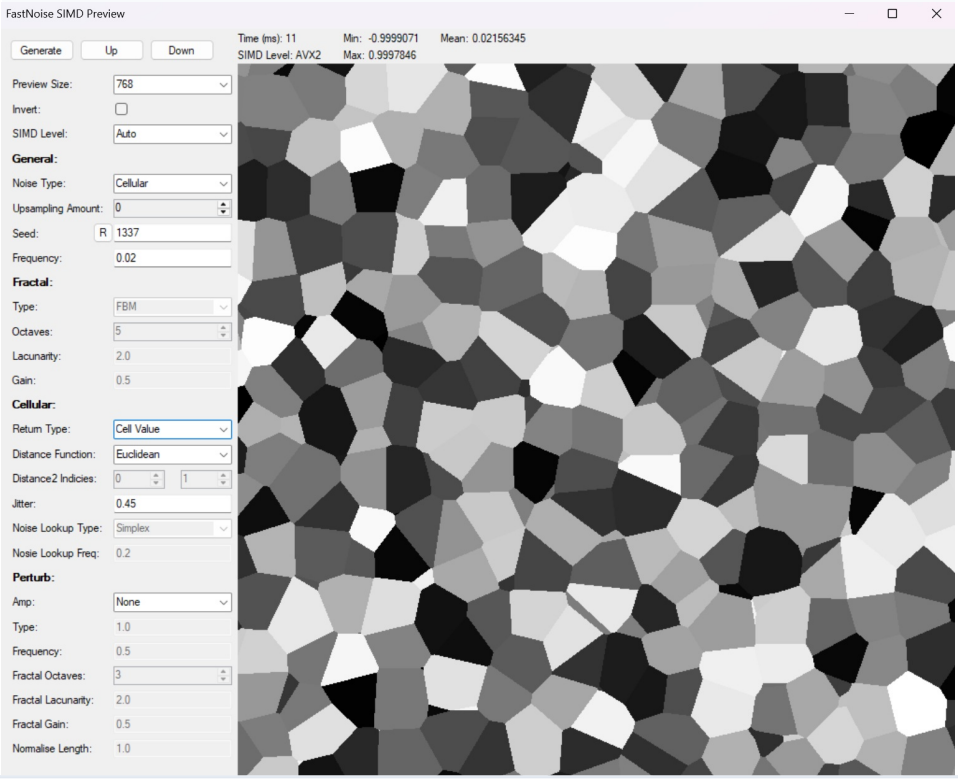
- Divides space into regions (cells).
- Produces hard boundaries.
- Useful for **biomes, caves, regions**.

Changing the return type or distance function alters the shape significantly.

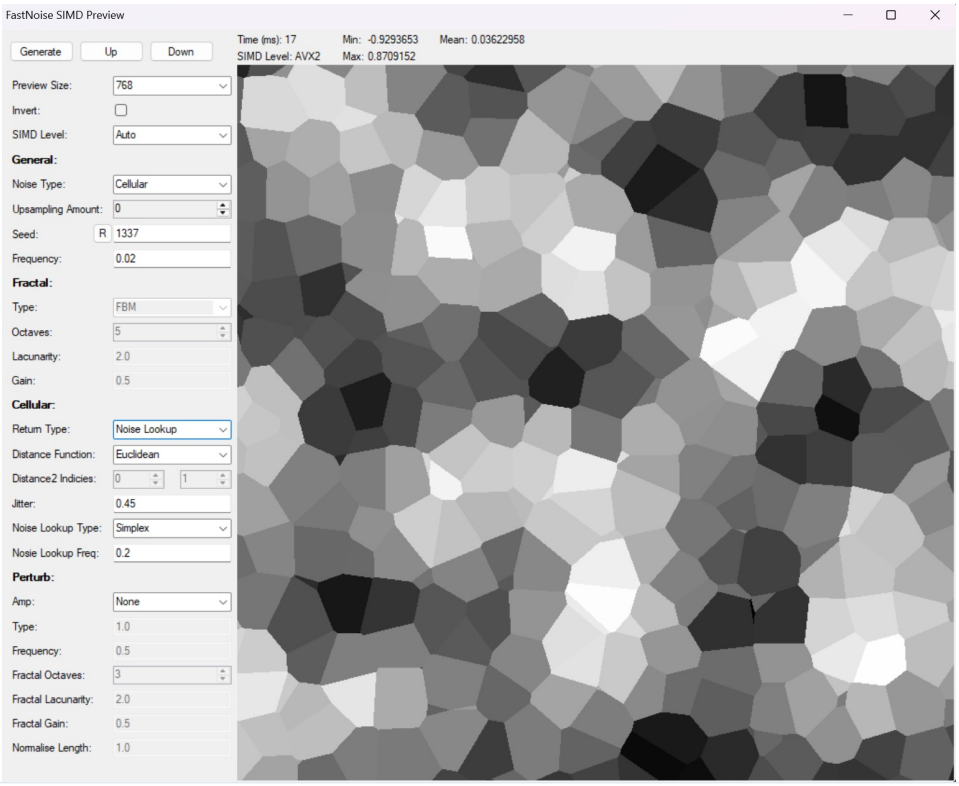
Cellular with Return Type: Distance



Cellular with Return Type: Cell Value



Cellular with Return Type: Noise Lookup



FREQUENCY

Tips

- Controls the **scale / zoom** of the noise.
- Lower frequency → larger features.
- Higher frequency → smaller details.

NOISE POST-PROCESSING TRICKS

Tips

- **Range conversion:** many libraries output noise in **[-1, 1]**. Convert to **[0, 1]** with: `(n + 1) * 0.5`
- **Squaring the noise:** `n = n * n` biases values toward 0, creating flatter low areas and sharper peaks.
- **Thresholding:** simple comparisons can create caves or masks: `n < 0.1`
- **Band selection:** select a slice of noise values: `n > 0.4 && n < 0.6`
- **Layering:** most interesting results come from combining multiple noises, not a single one.

Noise alone is rarely enough — shaping, filtering, and combining it is where the real control comes from!

LINEAR INTERPOLATION (LERP)

Tips

- **Lerp** blends between two values.
- The third parameter controls how much of each value you get. It should be between 0 and 1!
- Commonly used to **blend noises**, heights, colors, or biomes.

```
float lerp(float a, float b, float t)
  {
    return a + t * (b - a);
  }
```

- `t = 0` → returns `a`
- `t = 1` → returns `b`
- `t = 0.5` → halfway between

A common pattern is to blend two noises using a third one:

```
float result = lerp(noiseA, noiseB, controlNoise);
```

This is useful for smooth biome transitions, terrain variation, or mixing different cave and height patterns.